EcoFlux — Pipeline CI/CD e Containerização

Integrantes

- Alana de Menezes Pereira — RM 562562
- Carlos Eduardo Gonçalves de França — RM 561206
- Gustavo Cruz de Oliveira — RM 565633
- Nelson Trindade Teixeira — RM 563946
- Wendel de Santana Santos — RM 566354

1. Visão geral

O EcoFlux é uma API de gestão automatizada de resíduos. A aplicação registra pontos de coleta, descartes e coletas, mantendo alertas de capacidade para apoiar decisões operacionais ligadas à sustentabilidade.

A versão utilizada nesta entrega é a aplicação Java Spring Boot com Oracle Database XE. O projeto possui migrações Flyway, autenticação JWT e regras de negócio para impedir inconsistências no inventário.

2. Arquitetura da solução



3. Pipeline

O workflow `.github/workflows/ci-cd.yml` executa as seguintes etapas:

1. Checkout do código.
2. Configuração do Java 17.
3. Execução dos testes com `./mvnw -B test`.
4. Geração do JAR.
5. Construção e publicação da imagem no GitHub Container Registry em pushes para `main` ou em tags de versão.
6. Deploy de staging após push na branch `main`.
7. Deploy de produção após uma tag no formato `vX.Y.Z`.

A separação entre staging e produção é feita pelos ambientes do GitHub Actions. Cada ambiente deve ter seus próprios secrets e pode possuir regras de aprovação configuradas no repositório.

4. Testes automatizados

O pipeline não depende de um Oracle externo para validar o código. O perfil `test` usa H2 em memória e desabilita as migrações Flyway específicas do Oracle.

Foram cobertos:

- carregamento do contexto Spring;
- aumento do inventário ao registrar descarte;
- geração de alerta ao atingir 85% da capacidade;
- bloqueio de descarte acima da capacidade;
- bloqueio de descarte em ponto inativo;
- redução do inventário após coleta;
- resolução dos alertas pendentes após coleta.

Comando usado:

```
./mvnw -B test
```

5. Docker

O Dockerfile usa build em duas etapas:

```
FROM maven:3.9.9-eclipse-temurin-17 AS build
WORKDIR /app
COPY pom.xml .
RUN mvn dependency:go-offline -B
COPY src ./src
RUN mvn clean package -DskipTests -B

FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
RUN addgroup -S ecoflux && adduser -S ecoflux -G ecoflux
COPY --from=build /app/target/ecoflux-0.0.1-SNAPSHOT.jar app.jar
USER ecoflux
EXPOSE 8080
ENTRYPOINT ["java", "-jar", "app.jar"]
```

A imagem final não contém o código-fonte nem roda como root. As credenciais são fornecidas pelo ambiente, não pelo Dockerfile.

6. Docker Compose

O Compose cria dois serviços:

- `oracle-db`: Oracle XE com healthcheck e volume `oracle-data`;
- `ecoflux-app`: API Spring Boot, dependente da saúde do banco.

A comunicação ocorre pela rede `ecoflux-network`. As portas e os valores de ambiente podem ser alterados no `.env` a partir do modelo `.env.example`.

Comandos:

```
cp .env.example .env
# editar .env

docker compose up --build -d
docker compose ps
docker compose logs -f ecoflux-app
docker compose down
```

7. Deploy remoto

Para o deploy remoto, o servidor precisa ter Docker Engine, Docker Compose, acesso de saída ao GHCR e uma chave SSH configurada no GitHub.

Secrets esperados nos ambientes `staging` e `production`:

- `HOST`
- `USER`
- `SSH_KEY`
- `APP_DIR`
- `ORACLE_SYS_PASSWORD`
- `ORACLE_USER`
- `ORACLE_PASSWORD`

- `JWT_SECRET`
- `GHCR_READ_TOKEN`

O job remoto baixa o `docker-compose.yml` da mesma revisão publicada, cria o `.env` protegido e executa `docker compose pull` seguido de `docker compose up -d --no-build`.

Para que o primeiro push não tente acessar servidores ainda não configurados, os jobs de deploy dependem das variáveis `ENABLE_STAGING=true` e `ENABLE_PRODUCTION=true`, cadastradas nos respectivos ambientes do GitHub. Sem essas variáveis, build, testes e publicação da imagem continuam funcionando, mas os deploys são pulados.

8. Evidências obrigatórias

Esta seção deve receber as evidências reais depois que o repositório e os servidores forem configurados. Não devem ser utilizados prints simulados.

8.1 Build e testes

- URL da execução do GitHub Actions: `PREENCHER após publicar o repositório`
- Validação local: `BUILD SUCCESS`
- Resultado dos testes locais: `5 testes executados, 0 falhas, 0 erros`
- Resultado do job `Build e testes`: `PREENCHER após a primeira execução no GitHub Actions`

8.2 Imagem Docker

- URL da imagem no GHCR: `PREENCHER`
- Tag utilizada: `PREENCHER`

8.3 Staging

- URL ou IP do ambiente: `PREENCHER`
- URL da execução do deploy: `PREENCHER`
- Endpoint validado: `PREENCHER`
- Print do job e do ambiente: inserir aqui

8.4 Produção

- URL ou IP do ambiente: `PREENCHER`
- Tag de release: `PREENCHER`
- URL da execução do deploy: `PREENCHER`
- Endpoint validado: `PREENCHER`
- Print do job e do ambiente: inserir aqui

9. Dificuldades e decisões

Oracle no teste automatizado

A aplicação usa Oracle em produção, mas depender de um banco externo tornaria o pipeline instável. Por isso, o perfil de testes usa H2 em memória e mantém o Oracle para a execução integrada via Compose.

Segredos no repositório

As senhas que existiam nos arquivos iniciais foram retiradas do Dockerfile e do Compose. O repositório contém somente `.env.example`; os valores reais devem ficar nos secrets do GitHub ou no `.env` local.

Dois ambientes

Staging e produção possuem jobs separados. O staging acompanha a branch `main`; a produção é disparada por tag de versão e pode ser protegida com aprovação no GitHub.

10. Checklist de entrega

Item	Situação
Projeto compactado em `.zip`	Preparado
Dockerfile funcional	Preparado
`docker-compose.yml`	Preparado
Pipeline de build	Preparado
Pipeline de testes	Preparado
Deploy automatizado em staging	Configurado; requer servidor e secrets
Deploy automatizado em produção	Configurado; requer servidor e secrets
README.md	Preparado
Documentação técnica	Este documento
Evidências reais de staging e produção	Pendente de execução

> A configuração de CI/CD está pronta para execução, mas staging e produção não são declarados como concluídos sem servidores, credenciais e URLs verificáveis.